

PRODUCTION-GRADE MERN STACK APPLICATION DEPLOYMENT USING KUBERNETES, JENKINS AND ELK STACK

~ Moin Mohammed Naik

Technologies Used

- Technologies Used
- React.js
- Node.js
- Express.js
- MongoDB Replica Set
- Docker
- Kubernetes
- Jenkins
- NGINX Ingress Controller
- Cert Manager
- Elasticsearch
- Kibana
- Filebeat
- Metricbeat
- Elastic APM

Date: 16-06-2026

TABLE OF CONTENTS

Sr. No.	Title	Page No.
1.	Introduction	3
2.	Chapter 1: Project Overview	4
3.	Chapter 2: Technology Stack	6
4.	Chapter 3: System Architecture	8
5.	Chapter 4: Kubernetes Implementation	12
7.	Chapter 5: CI/CD Pipeline Implementation	17
8.	Chapter 6: Monitoring and Observability	22
9.	Chapter 7: Result and Conclusion	28

INTRODUCTION

Modern applications require scalable infrastructure, automated deployment mechanisms, centralized monitoring, and high availability to meet production standards. Traditional deployment methods often involve manual configuration, downtime during updates, and limited visibility into application performance.

To address these challenges, organizations adopt DevOps practices and cloud-native technologies such as Docker, Kubernetes, Jenkins, and Elasticsearch. These tools enable automation, scalability, observability, and efficient resource utilization.

This project focuses on designing and deploying a Production-Grade MERN Stack Application using industry-standard DevOps practices. The application consists of a React.js frontend, Node.js and Express.js backend, and MongoDB database deployed as a Replica Set for high availability.

The complete infrastructure is containerized using Docker and orchestrated using Kubernetes. Continuous Integration and Continuous Deployment (CI/CD) pipelines are implemented using Jenkins, allowing automated application builds, testing, image creation, and deployment.

To provide complete observability, the Elastic Stack (ELK) is integrated into the architecture. Filebeat collects application logs, Metricbeat collects infrastructure metrics, and Elastic APM provides application performance monitoring. All collected data is centralized in Elasticsearch and visualized through Kibana dashboards.

Additionally, NGINX Ingress Controller and Let's Encrypt SSL certificates are configured to provide secure HTTPS communication between users and the application.

The final solution demonstrates a fully automated, scalable, secure, and observable production-grade deployment capable of supporting real-world workloads.

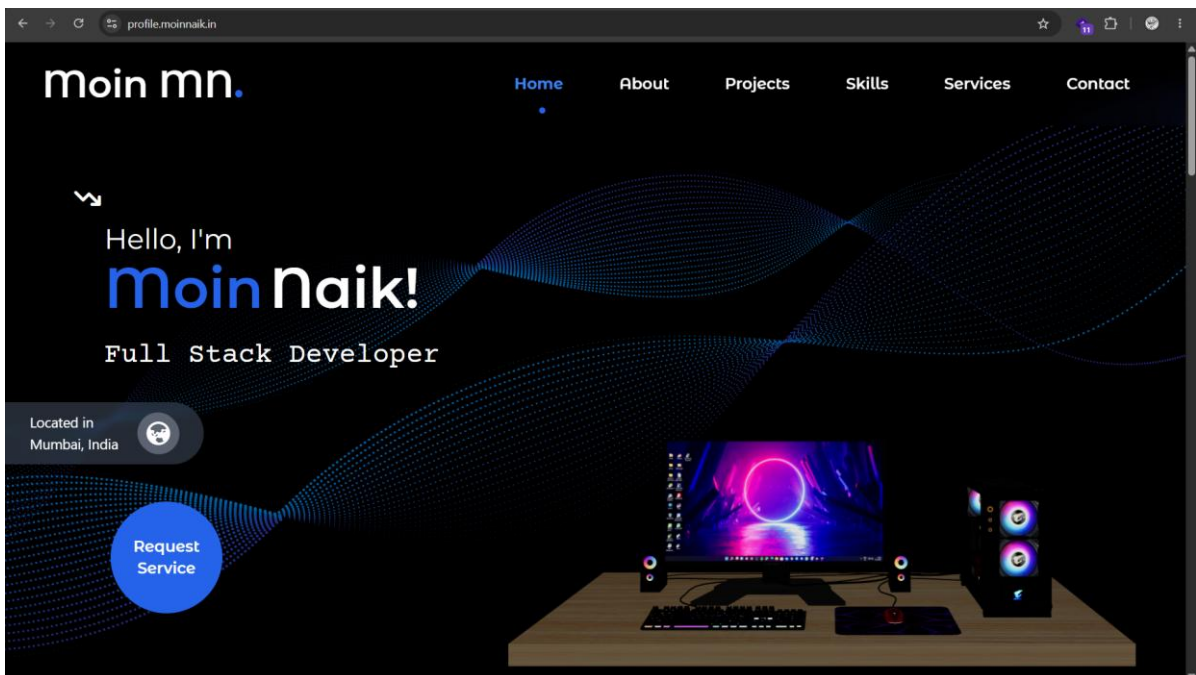


Figure 1: Deployed Application

CHAPTER 1: PROJECT OVERVIEW

This project is a Production-Grade MERN Stack Application deployed on Kubernetes using DevOps best practices. The application consists of a React frontend, Node.js/Express backend, and MongoDB Replica Set database.

The entire application is containerized using Docker and deployed inside a Kubernetes cluster for scalability and high availability. Jenkins is used to automate the CI/CD process, enabling automatic build, image creation, testing, and deployment whenever code changes are pushed to GitHub.

To ensure database reliability, MongoDB is configured as a 3-node Replica Set with one Primary and two Secondary nodes. This provides redundancy and protects against data loss in case of node failure.

An NGINX Ingress Controller is used to expose the application externally, while Let's Encrypt SSL certificates secure communication through HTTPS.

For monitoring and observability, the ELK Stack is integrated into the environment. Filebeat collects logs from application containers, Metricbeat gathers infrastructure metrics, and Elastic APM monitors backend application performance. All monitoring data is stored in Elasticsearch and visualized through Kibana dashboards.

Key Features

- MERN Stack Application
- Docker Containerization
- Kubernetes Deployment
- MongoDB Replica Set
- Jenkins CI/CD Pipeline
- HTTPS using Let's Encrypt SSL
- NGINX Ingress Controller
- Horizontal Pod Autoscaling (HPA)
- Centralized Logging using ELK
- Application Performance Monitoring (APM)
- High Availability Architecture

Project Workflow

1. Developer pushes code to GitHub.
2. Jenkins automatically triggers the pipeline.
3. Docker images are built and pushed to Docker Hub.
4. Images are distributed to Kubernetes worker nodes.
5. Kubernetes deploys updated application containers.
6. MongoDB Replica Set stores application data.
7. Users access the application through the Ingress Controller.
8. Logs, metrics, and traces are collected by the ELK Stack for monitoring.

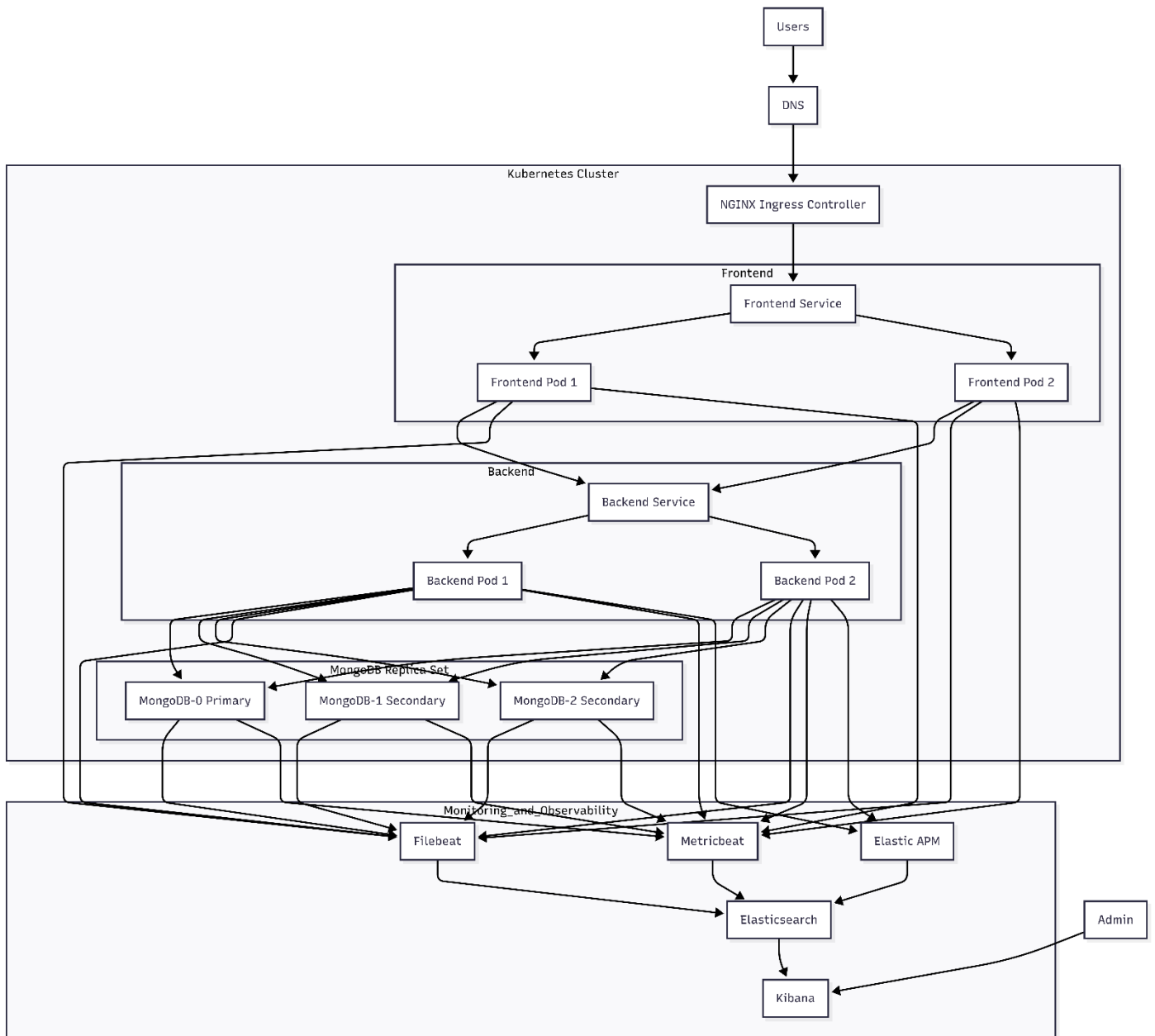


Figure 2: Overall System Architecture

CHAPTER 2: TECHNOLOGY STACK

The project uses modern DevOps and cloud-native technologies to build, deploy, monitor, and manage the application.

Component	Technology Used	Purpose
Frontend	React.js + Vite	User Interface
Backend	Node.js + Express.js	REST API Services
Database	MongoDB Replica Set	Data Storage & High Availability
Containerization	Docker	Application Packaging
Orchestration	Kubernetes	Container Management
CI/CD	Jenkins	Automated Build & Deployment
Image Registry	Docker Hub	Image Storage
Ingress	NGINX Ingress Controller	External Access
SSL	Cert Manager + Let's Encrypt	HTTPS Security
Logging	Filebeat	Log Collection
Monitoring	Metricbeat	Infrastructure Monitoring
APM	Elastic APM	Application Performance Monitoring
Analytics	Elasticsearch	Data Storage & Search
Visualization	Kibana	Monitoring Dashboards
Source Control	GitHub	Code Repository

Infrastructure Details

The Kubernetes environment consists of:

- 1 Master Node
- 2 Worker Nodes
- 1 Jenkins Server
- 1 ELK Server

The application components are deployed as follows:

- Frontend Deployment – 2 Replicas
- Backend Deployment – 2 Replicas
- MongoDB Replica Set – 3 Pods

- Horizontal Pod Autoscaler (HPA) enabled for Backend
- NGINX Ingress Controller for external traffic routing

This architecture provides scalability, fault tolerance, automated deployment, centralized logging, and monitoring capabilities suitable for production workloads.

```

root@k8s-master: ~
root@k8s-master:~# kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE      KERNEL-VERSION   CONTAINER-RUNTIME
k8s-master    Ready     control-plane  22d   v1.30.14   10.0.1.170    <none>        Ubuntu 26.04 LTS  7.0.0-1006-aws   containerd://2.2.4
worker-node-1 Ready     <none>     22d   v1.30.14   10.0.2.145    <none>        Ubuntu 26.04 LTS  7.0.0-1004-aws   containerd://2.2.4
worker-node-2 Ready     <none>     22d   v1.30.14   10.0.2.151    <none>        Ubuntu 26.04 LTS  7.0.0-1004-aws   containerd://2.2.4
worker-node-3 Ready     <none>     22d   v1.30.14   10.0.2.195    <none>        Ubuntu 26.04 LTS  7.0.0-1004-aws   containerd://2.2.4
worker-node-4 Ready     <none>     22d   v1.30.14   10.0.2.66     <none>        Ubuntu 26.04 LTS  7.0.0-1004-aws   containerd://2.2.4
root@k8s-master:~# kubectl get all
NAME                READY   STATUS    RESTARTS   AGE
pod/backend-555c756b5d-4vxfg  1/1     Running   1 (6m27s ago)    5d21h
pod/backend-555c756b5d-gvtz2  1/1     Running   1 (6m24s ago)    5d21h
pod/frontend-68d957477-cd6xv  1/1     Running   1 (6m24s ago)    5d21h
pod/frontend-68d957477-tvbpf  1/1     Running   1 (6m27s ago)    5d21h
pod/mongodb-0              1/1     Running   16 (6m23s ago)   13d
pod/mongodb-1              1/1     Running   5 (6m25s ago)    8d
pod/mongodb-2              1/1     Running   16 (6m22s ago)   13d

NAME                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/backend-service  ClusterIP     10.97.133.133 <none>        4518/TCP   16d
service/frontend-service ClusterIP     10.111.18.122 <none>        80/TCP     16d
service/mongodb          ClusterIP     None          <none>        27017/TCP  16d

NAME                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/backend  2/2     2             2           16d
deployment.apps/frontend 2/2     2             2           16d

NAME                DESIRED   CURRENT   READY   AGE
replicaset.apps/backend-555c756b5d  2         2         2       5d21h
replicaset.apps/backend-5995c8bcb8  0         0         0       5d23h
replicaset.apps/backend-6c6b967d68  0         0         0       5d22h
replicaset.apps/backend-b57f6cc66   0         0         0       5d22h
replicaset.apps/frontend-5d4f8564cd  0         0         0       5d21h
replicaset.apps/frontend-68d957477   2         2         2       5d21h
replicaset.apps/frontend-6c4944b948  0         0         0       5d22h
replicaset.apps/frontend-6dd597b46d  0         0         0       6d1h

NAME                READY   AGE
statefulset.apps/mongodb  3/3     16d

NAME                REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/backend-hpa  Deployment/backend        memory: <unknown>/90%  2         4         2          10d
horizontalpodautoscaler.autoscaling/frontend-hpa  Deployment/frontend        memory: <unknown>/90%  2         4         2          5d21h
root@k8s-master:~#

```

Figure 3: Kubernetes Cluster Nodes & Resources

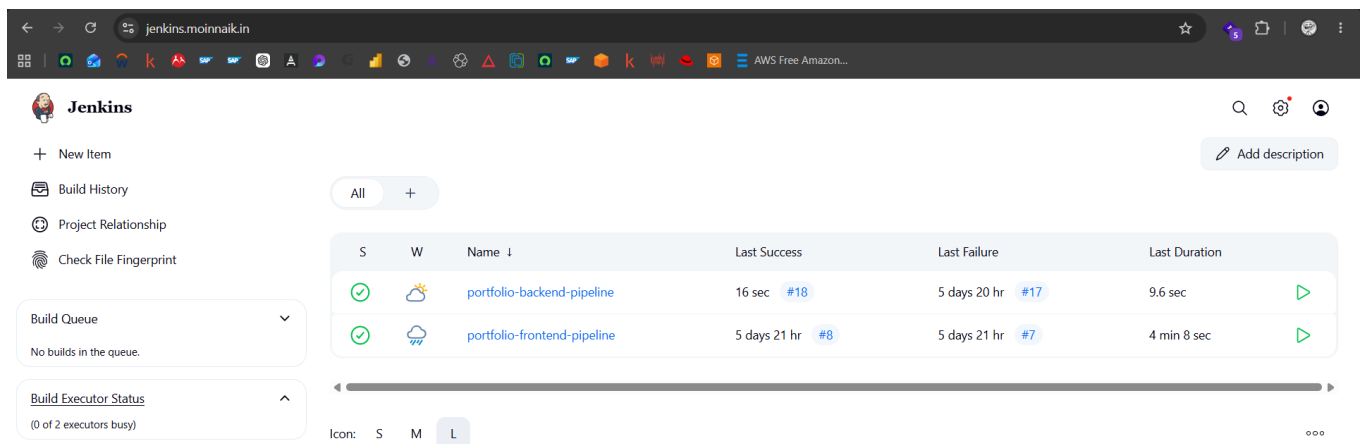


Figure 4: Jenkins Dashboard

CHAPTER 3: SYSTEM ARCHITECTURE

The application follows a microservice-based architecture deployed on Kubernetes. All application components are containerized using Docker and managed through Kubernetes Deployments and StatefulSets.

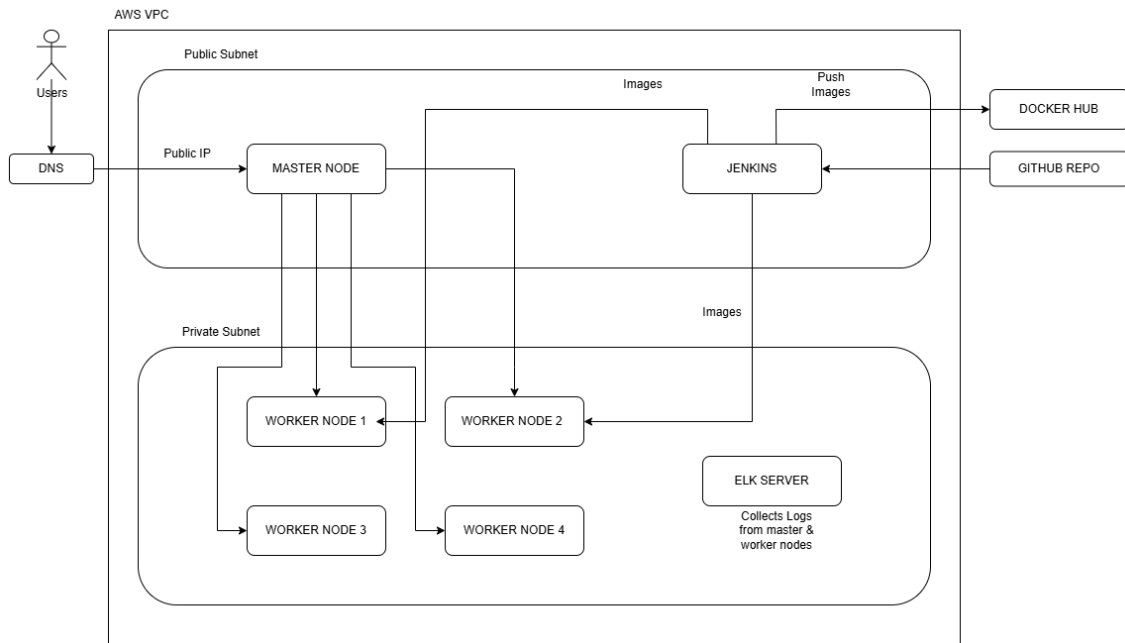


Figure 5: Overall Production Architecture

The architecture consists of the following layers:

4.1 Client Layer

Users access the application through the domain:

<https://profile.moinnaik.in>

All incoming traffic is routed through the NGINX Ingress Controller, which acts as the entry point to the Kubernetes cluster.

4.2 Frontend Layer

The frontend is developed using React.js and deployed as a Kubernetes Deployment with multiple replicas.

Responsibilities:

- Display portfolio information
- Handle user interactions
- Communicate with backend APIs
- Serve static web content

Frontend traffic is distributed across multiple pods using a Kubernetes Service.

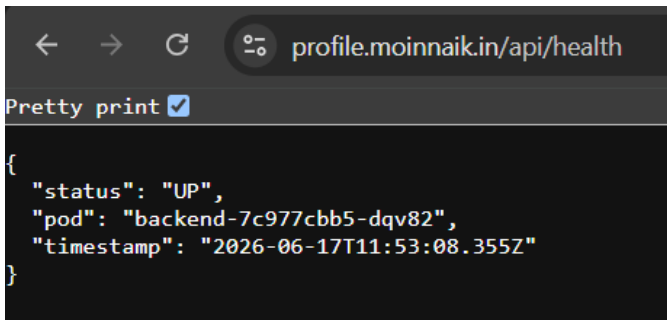
4.3 Backend Layer

The backend is developed using Node.js and Express.js.

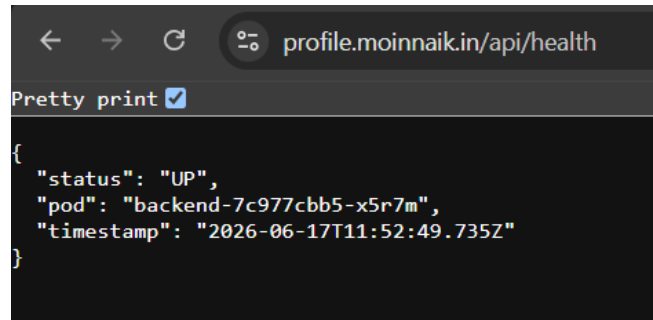
Responsibilities:

- Process API requests
- Handle business logic
- Manage database operations
- Provide REST APIs to frontend applications

Backend pods are deployed with multiple replicas to ensure high availability and fault tolerance.



A screenshot of a web browser window showing the URL `profile.moinnaik.in/api/health`. The browser's developer tools are open, displaying the response of the health endpoint. The response is a JSON object with the following fields: `"status": "UP"`, `"pod": "backend-7c977cbb5-dqv82"`, and `"timestamp": "2026-06-17T11:53:08.355Z"`.



A second screenshot of a web browser window showing the same URL `profile.moinnaik.in/api/health`. The response is a JSON object with the following fields: `"status": "UP"`, `"pod": "backend-7c977cbb5-x5r7m"`, and `"timestamp": "2026-06-17T11:52:49.735Z"`.

4.4 Database Layer

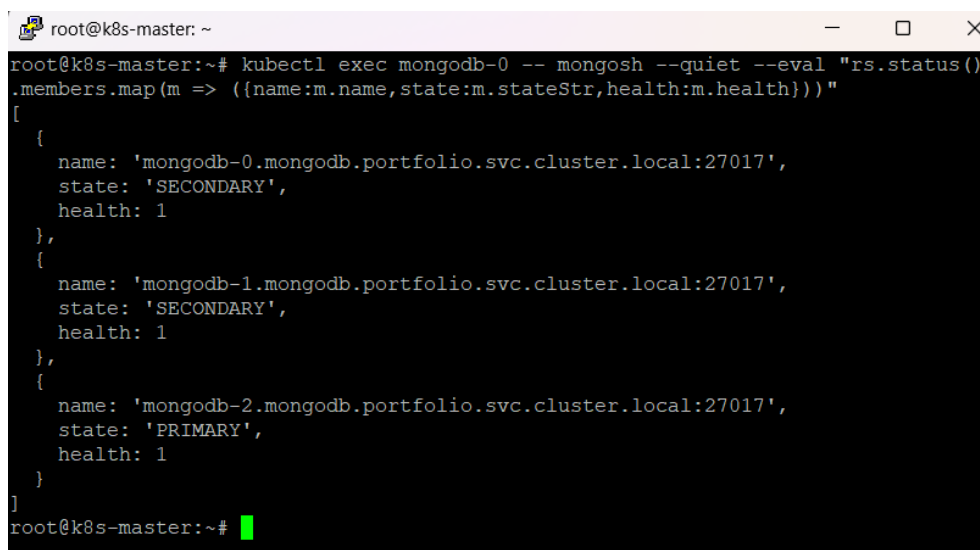
MongoDB is deployed as a StatefulSet with a Replica Set configuration consisting of:

- MongoDB-0 (Primary)
- MongoDB-1 (Secondary)
- MongoDB-2 (Secondary)

Benefits:

- High Availability
- Data Replication
- Automatic Failover
- Data Consistency

If the Primary node fails, one of the Secondary nodes automatically becomes the new Primary.



A terminal window screenshot showing the command `kubectl exec mongodb-0 -- mongosh --quiet --eval "rs.status().members.map(m => ({name:m.name,state:m.stateStr,health:m.health}))"` being executed. The output shows the status of the MongoDB Replica Set, listing three members: `'mongodb-0.mongodb.portfolio.svc.cluster.local:27017'` (SECONDARY, health: 1), `'mongodb-1.mongodb.portfolio.svc.cluster.local:27017'` (SECONDARY, health: 1), and `'mongodb-2.mongodb.portfolio.svc.cluster.local:27017'` (PRIMARY, health: 1).

Figure 6: MongoDB Replica Set Status

4.5 CI/CD Layer

Jenkins automates the application deployment process.

Pipeline Workflow:

1. Developer pushes code to GitHub.
2. Jenkins pipeline starts automatically.
3. Docker image is built.
4. Application health checks are performed.
5. Image is pushed to Docker Hub.
6. Image tar file is distributed to worker nodes.
7. Kubernetes deployment is updated.
8. Rolling update is performed with zero downtime.

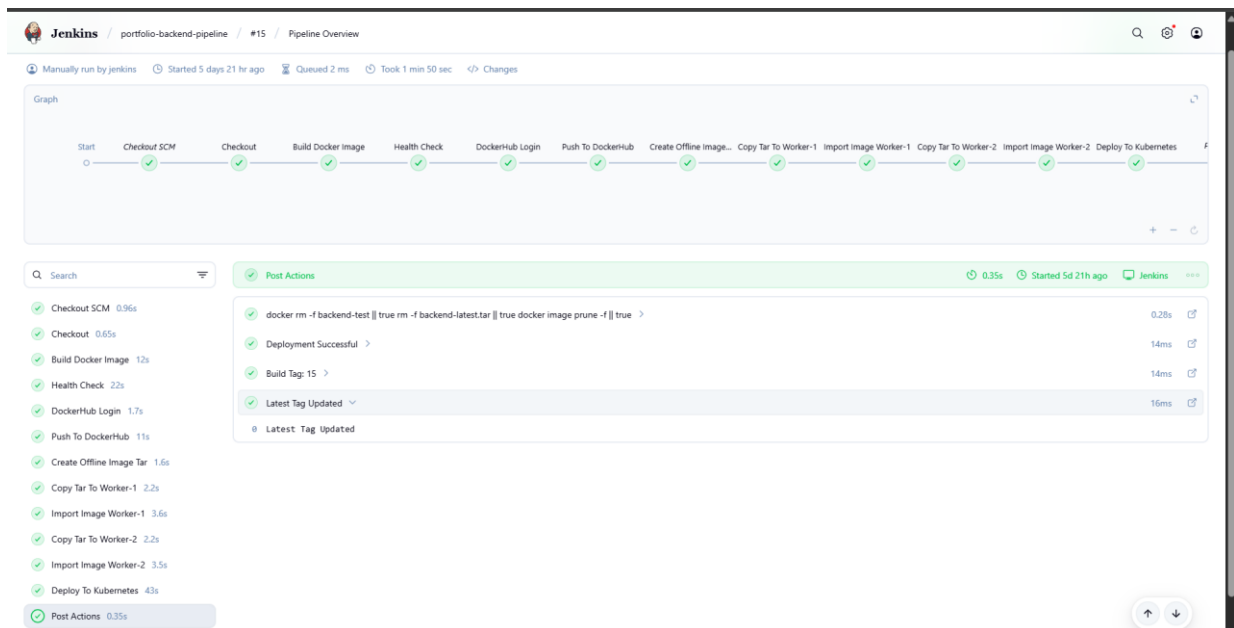


Figure 7: Jenkins CI/CD Pipeline

4.6 Monitoring Layer

The ELK Stack provides centralized monitoring and observability.

Filebeat

Collects logs from:

- Frontend Pods
- Backend Pods
- MongoDB Pods

Metricbeat

Collects:

- CPU Usage
- Memory Usage
- Disk Utilization

- Kubernetes Metrics

Elastic APM

Monitors:

- API Response Time
- Request Traces
- Application Performance
- Error Tracking

Elasticsearch

Stores logs, metrics, and APM traces.

Kibana

Provides dashboards for:

- Log Analysis
- Infrastructure Monitoring
- Application Monitoring
- Troubleshooting

4.7 Security Layer

The application is secured using:

- HTTPS Encryption
- Let's Encrypt SSL Certificates
- Kubernetes Secrets
- Ingress Security Rules

This ensures secure communication between users and application services.

CHAPTER 4: KUBERNETES IMPLEMENTATION

The application is deployed on a Kubernetes cluster to provide scalability, high availability, self-healing, and automated workload management.

The deployment consists of Frontend, Backend, MongoDB Replica Set, Services, Ingress Controller, and Horizontal Pod Autoscaler.

5.1 Namespace Configuration

A dedicated namespace named **portfolio** is created to isolate application resources from other workloads running in the cluster.

Benefits:

- Resource isolation
- Easier management
- Simplified monitoring

```
root@k8s-master: ~  
root@k8s-master:~# kubectl get all -n portfolio  
NAME                                READY   STATUS    RESTARTS   AGE  
pod/backend-555c756b5d-4vxfg        1/1     Running   2 (8m53s ago)    5d23h  
pod/backend-555c756b5d-qvtz2        1/1     Running   2 (8m52s ago)    5d23h  
pod/frontend-68d957477-cd6xv        1/1     Running   2 (8m52s ago)    5d22h  
pod/frontend-68d957477-tvbpf        1/1     Running   2 (8m53s ago)    5d22h  
pod/mongodb-0                       1/1     Running   17 (8m52s ago)   14d  
pod/mongodb-1                       1/1     Running   6 (8m53s ago)    8d  
pod/mongodb-2                       1/1     Running   17 (8m52s ago)   14d  
  
NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE  
service/backend-service             ClusterIP          10.97.133.133    <none>            4518/TCP          17d  
service/frontend-service            ClusterIP          10.111.18.122    <none>            80/TCP            17d  
service/mongodb                     ClusterIP          None             <none>            27017/TCP         17d  
  
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE  
deployment.apps/backend             2/2     2             2            17d  
deployment.apps/frontend            2/2     2             2            17d  
  
NAME                                DESIRED   CURRENT   READY   AGE  
replicaset.apps/backend-555c756b5d  2         2         2        5d23h  
replicaset.apps/backend-5995c8bcb8  0         0         0        6d  
replicaset.apps/backend-6cfb967d68  0         0         0        6d  
replicaset.apps/backend-b57f6cc66   0         0         0        6d  
replicaset.apps/frontend-5d4f8564cd  0         0         0        5d23h  
replicaset.apps/frontend-68d957477  2         2         2        5d22h  
replicaset.apps/frontend-6c4944b948  0         0         0        5d23h  
replicaset.apps/frontend-6dd597b46d  0         0         0        6d2h  
  
NAME                                READY   AGE  
statefulset.apps/mongodb            3/3     17d  
  
NAME                                REFERENCE           TARGETS          MINPODS   MAXPODS   REPLICAS   AGE  
horizontalpodautoscaler.autoscaling/backend-hpa  Deployment/backend  memory: <unknown>/90%  2         4         2           10d  
horizontalpodautoscaler.autoscaling/frontend-hpa  Deployment/frontend  memory: <unknown>/90%  2         4         2           5d23h  
root@k8s-master:~#
```

Figure 8: Namespace Resources

5.2 Frontend Deployment

The React frontend application is deployed using a Kubernetes Deployment.

Configuration:

- 2 Replicas
- Docker Image from Docker Hub
- Rolling Update Strategy
- ClusterIP Service

Functions:

- Serve static web content
- Communicate with backend APIs
- Provide user interface

5.3 Backend Deployment

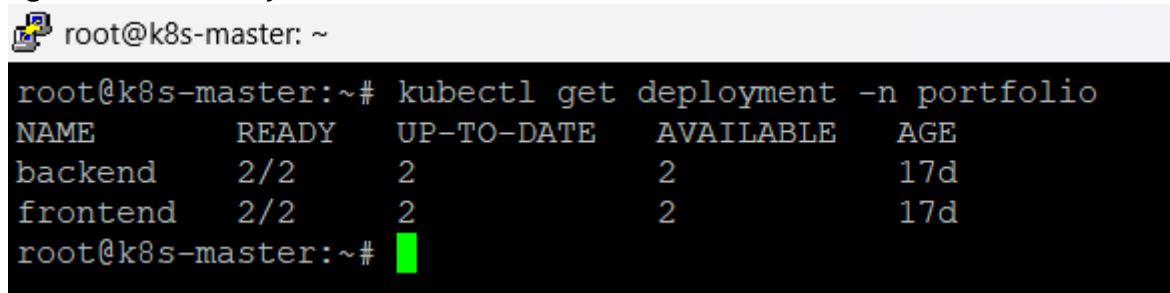
The Node.js backend application is deployed using a Kubernetes Deployment.

Configuration:

- 2 Replicas
- Docker Image from Docker Hub
- Health Checks
- Rolling Updates
- ClusterIP Service

Functions:

- API Processing
- Business Logic Execution
- MongoDB Connectivity



```
root@k8s-master: ~  
root@k8s-master:~# kubectl get deployment -n portfolio  
NAME          READY   UP-TO-DATE   AVAILABLE   AGE  
backend       2/2     2            2           17d  
frontend      2/2     2            2           17d  
root@k8s-master:~#
```

Figure 9: Deployments

5.4 MongoDB StatefulSet

MongoDB is deployed using StatefulSets because database workloads require stable identities and persistent storage.

Configuration:

- 3 Replica Set Members
- Persistent Volumes
- Persistent Volume Claims
- Automatic Recovery

Replica Set Members:

- mongodb-0 (Primary)
- mongodb-1 (Secondary)
- mongodb-2 (Secondary)

```
root@k8s-master: ~  
root@k8s-master:~# kubectl get statefulset -n portfolio  
NAME          READY    AGE  
mongodb       3/3      17d  
root@k8s-master:~#
```

Figure 10: MongoDB StatefulSet

5.5 Services

Kubernetes Services provide internal communication between components.

Frontend Service

Purpose:

- Load balance frontend traffic
- Expose frontend pods internally

Backend Service

Purpose:

- Load balance backend API requests
- Provide stable endpoint for frontend

MongoDB Headless Service

Purpose:

- Enable Replica Set communication
- Provide stable DNS records

```
root@k8s-master: ~  
root@k8s-master:~# kubectl get svc -n portfolio  
NAME             TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE  
backend-service   ClusterIP     10.97.133.133 <none>         4518/TCP         17d  
frontend-service  ClusterIP     10.111.18.122 <none>         80/TCP           17d  
mongodb           ClusterIP     None          <none>         27017/TCP        17d  
root@k8s-master:~#
```

Figure 11: Services

5.6 Ingress Configuration

NGINX Ingress Controller is used to expose the application externally.

Domain: profile.moinnaik.in

Responsibilities:

- Route incoming traffic
- SSL Termination
- HTTPS Redirection

- Load Balancing

```

root@k8s-master: ~
root@k8s-master:~# kubectl get ingress -n portfolio
kubectl describe ingress portfolio-ingress -n portfolio
NAME          CLASS  HOSTS          ADDRESS  PORTS  AGE
portfolio-ingress  nginx  profile.moinnaik.in  80, 443  17d
Name:          portfolio-ingress
Labels:        <none>
Namespace:     portfolio
Address:
Ingress Class:  nginx
Default backend: <default>
TLS:
  portfolio-tls terminates profile.moinnaik.in
Rules:
  Host            Path  Backends
  ----            -
  profile.moinnaik.in  /    frontend-service:80 (192.168.19.163:80,192.168.212.98:80)
Annotations:
  cert-manager.io/cluster-issuer: letsencrypt-prod
  nginx.ingress.kubernetes.io/ssl-redirect: true
Events:
  Type    Reason    Age   From                      Message
  ----    -
  Normal  Sync      112m  nginx-ingress-controller  Scheduled for sync
  Normal  Sync      12m   nginx-ingress-controller  Scheduled for sync
root@k8s-master:~#

```

Figure 12: Ingress Configuration

5.7 SSL Certificate Management

Cert Manager is used for automatic certificate issuance and renewal.

Certificate Provider:

Let's Encrypt

Benefits:

- Automatic Certificate Renewal
- Secure HTTPS Communication
- Reduced Manual Administration

```

root@k8s-master: ~
root@k8s-master:~# kubectl get certificate -n portfolio
NAME          READY  SECRET          AGE
portfolio-tls  True   portfolio-tls    11d
root@k8s-master:~#

```

Figure 13: SSL Certificate

5.8 Horizontal Pod Autoscaler

A Horizontal Pod Autoscaler (HPA) is configured for the backend deployment.

Configuration:

- Minimum Pods: 2
- Maximum Pods: 4
- Memory Utilization Threshold: 90%

Benefits:

- Automatic Scaling
- Better Resource Utilization
- Improved Application Availability

```
root@k8s-master: ~  
root@k8s-master:~# kubectl get hpa -n portfolio  
NAME           REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE  
backend-hpa     Deployment/backend        memory: <unknown>/90%  2         4         2          10d  
frontend-hpa    Deployment/frontend       memory: <unknown>/90%  2         4         2          5d23h  
root@k8s-master:~#
```

Figure 14: Horizontal Pod Autoscaler

5.9 Rolling Updates

Kubernetes Deployments use rolling updates during application deployment.

Advantages:

- Zero Downtime Deployment
- Automatic Rollback Support
- Continuous Availability

```
root@k8s-master: ~  
root@k8s-master:~# kubectl get pods -n portfolio -o wide  
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES  
backend-555c756b5d-4vxfg  1/1     Running   2 (14m ago)  5d23h  192.168.19.160  worker-node-2    <none>           <none>  
backend-555c756b5d-qvtz2  1/1     Running   2 (14m ago)  5d23h  192.168.212.94  worker-node-1    <none>           <none>  
frontend-68d957477-cd6xv  1/1     Running   2 (14m ago)  5d22h  192.168.212.98  worker-node-1    <none>           <none>  
frontend-68d957477-tvbpf  1/1     Running   2 (14m ago)  5d22h  192.168.19.163  worker-node-2    <none>           <none>  
mongodb-0              1/1     Running   17 (14m ago)  14d    192.168.8.249   worker-node-3    <none>           <none>  
mongodb-1              1/1     Running   6 (14m ago)   8d     192.168.200.166 worker-node-4    <none>           <none>  
mongodb-2              1/1     Running   17 (14m ago)  14d    192.168.8.252   worker-node-3    <none>           <none>  
root@k8s-master:~#
```

Figure 15: Running Pods

The CI/CD pipeline updates application containers without affecting end users.

CHAPTER 5: CI/CD PIPELINE IMPLEMENTATION

A Continuous Integration and Continuous Deployment (CI/CD) pipeline is implemented using Jenkins to automate the application build, testing, image creation, distribution, and deployment process.

The pipeline eliminates manual deployment steps and ensures consistent releases across the Kubernetes environment.

6.1 CI/CD Workflow

1. The deployment workflow follows the sequence below:
2. Developer pushes code to GitHub.
3. Jenkins automatically detects code changes.
4. Source code is pulled from the repository.
5. Docker image is built.
6. Application health checks are executed.
7. Docker image is pushed to Docker Hub.
8. Image tar files are generated.
9. Images are copied to Kubernetes worker nodes.
10. Images are imported into containerd.
11. Kubernetes deployment is updated.
12. Rolling update deploys the new version.
13. Application becomes available without downtime.

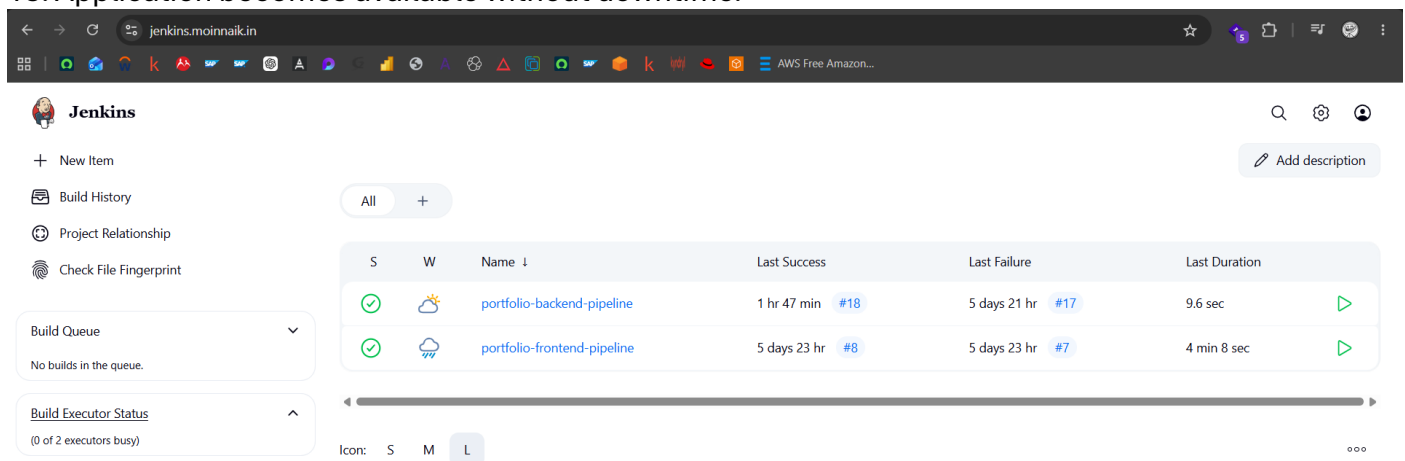


Figure 16: Jenkins Dashboard

6.2 Source Code Management

GitHub is used as the central source code repository.

Repositories include:

- Frontend Application

- Backend Application
- Kubernetes Manifests
- Jenkins Pipeline Definitions

Branch Strategy:

- main branch
- prod branch for production deployments

6.3 Backend Pipeline

The backend Jenkins pipeline performs the following actions:

Source Code Checkout

Jenkins pulls the latest code from GitHub.

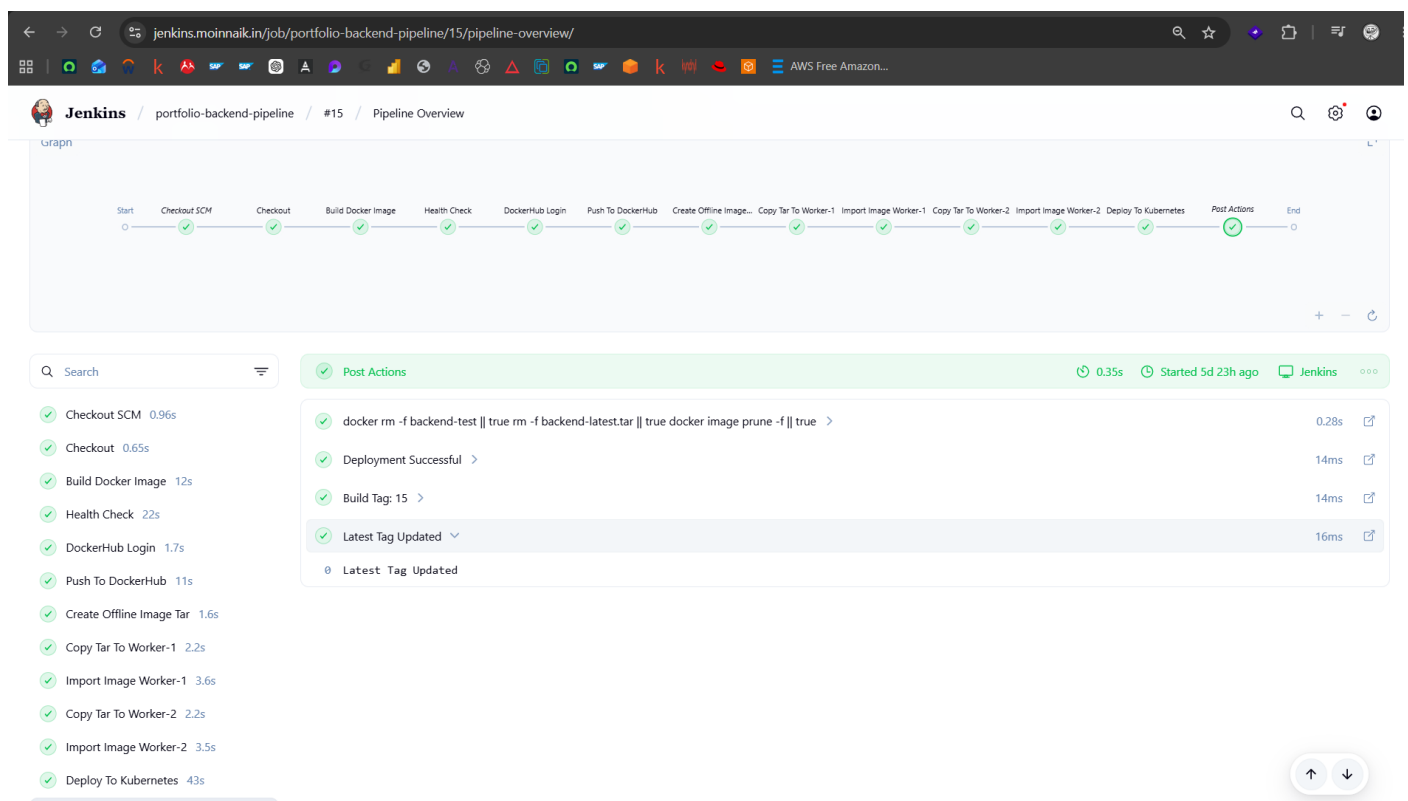


Figure 17: Backend Pipeline Success

Dependency Installation

Node.js dependencies are installed using npm.

Docker Image Build

Backend Docker image is created and tagged with:

- Build Number
- Latest

Example:

moinnaik/portfolio-backend:15

moinnaik/portfolio-backend

Health Verification

A temporary container is launched to verify application startup before deployment.

Docker Hub Push

The image is pushed to Docker Hub.

Offline Image Distribution

Docker image tar files are generated and transferred to Kubernetes worker nodes.

Kubernetes Deployment

The backend deployment is restarted to consume the latest image.

6.4 Frontend Pipeline

The frontend Jenkins pipeline follows a similar process:

Source Code Checkout

Latest frontend code is pulled from GitHub.

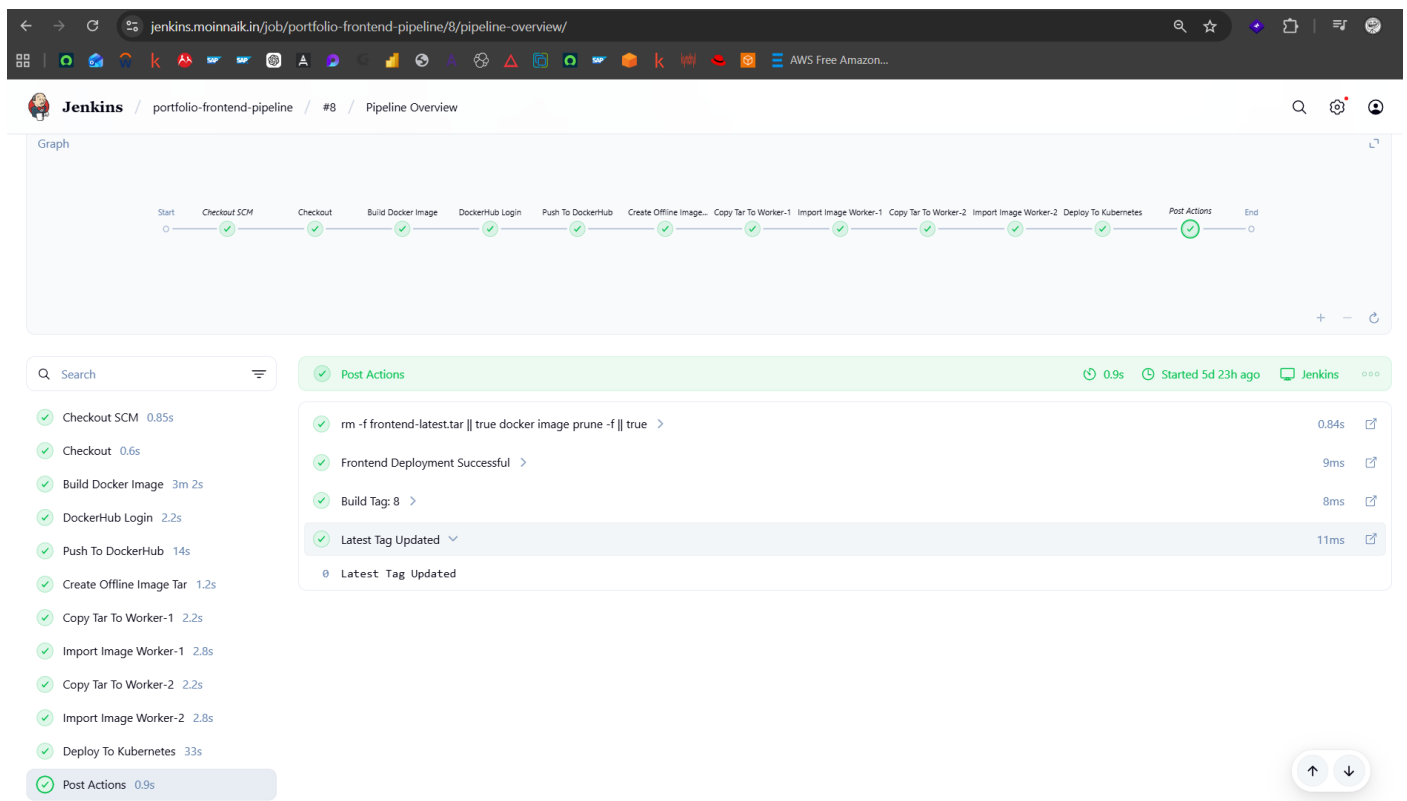


Figure 18: Frontend Pipeline Success

Dependency Installation

Required frontend packages are installed.

Production Build

The React application is compiled using Vite.

Docker Image Creation

Frontend image is tagged using:

- Build Number
- Latest

Example:

moinnaik/portfolio-frontend:15

moinnaik/portfolio-frontend

Docker Hub Push

Images are pushed to Docker Hub.

Worker Node Distribution

Image tar files are copied and imported into worker nodes.

Kubernetes Deployment

Frontend deployment is restarted to use the updated image.

6.5 Image Distribution Strategy

Worker nodes are configured without requiring direct internet access for image pulls.

To achieve this:

- Jenkins creates Docker image tar files.
- Tar files are copied using SCP.
- Images are imported using containerd.
- Kubernetes consumes locally available images.

Benefits:

- Faster deployment
- Reduced external dependency
- Improved security

6.6 Deployment Strategy

Rolling updates are used for application deployment.

Advantages:

- Zero Downtime Deployment
- Service Availability

- Automatic Pod Replacement
- Easy Rollback

6.7 Benefits of CI/CD Automation

- Faster Releases
- Reduced Human Error
- Consistent Deployments
- Automated Validation
- Improved Reliability
- Simplified Operations

CHAPTER 6: MONITORING AND OBSERVABILITY

To ensure application reliability, performance monitoring, and centralized log management, an ELK-based observability platform is implemented.

The monitoring solution consists of:

- Elasticsearch
- Kibana
- Filebeat
- Metricbeat
- Elastic APM

The platform provides centralized visibility into application logs, infrastructure metrics, and application performance.

7.1 Monitoring Architecture

The observability platform collects data from Kubernetes workloads and stores it centrally in Elasticsearch.

Collected data includes:

- Application Logs
- Container Logs
- Kubernetes Metrics
- Node Metrics
- API Performance Metrics
- Application Traces

All collected information is visualized through Kibana dashboards.

7.2 Elasticsearch

Elasticsearch serves as the central data storage engine.

Responsibilities:

- Store logs
- Store metrics
- Store APM traces
- Enable fast searching and filtering

Benefits:

- High-speed indexing
- Real-time search
- Scalability
- Centralized data management

7.3 Kibana

Kibana provides visualization and analysis capabilities.

Features Used:

- Discover
- Dashboards
- Infrastructure Monitoring
- APM Monitoring
- Log Analysis

Kibana enables administrators to monitor the health and performance of the entire application stack.

7.4 Filebeat

Filebeat is deployed to collect logs from Kubernetes workloads.

Log Sources:

- Frontend Pods
- Backend Pods
- MongoDB Pods
- Kubernetes System Components

Workflow:

1. Logs are generated by containers.
2. Filebeat collects logs.
3. Logs are forwarded to Elasticsearch.
4. Kibana displays logs in real time.

Benefits:

- Centralized Logging
- Faster Troubleshooting
- Real-Time Monitoring

7.5 Metricbeat

Metricbeat collects infrastructure and Kubernetes metrics.

Collected Metrics:

- CPU Usage
- Memory Usage
- Disk Usage
- Network Usage
- Pod Metrics
- Node Metrics

Benefits:

- Resource Monitoring
- Capacity Planning

- Performance Analysis

7.6 Elastic APM

Elastic APM is integrated with the Node.js backend application.

Monitored Parameters:

- API Response Time
- Request Throughput
- Error Rate
- Transaction Traces
- Service Dependencies

The APM agent automatically sends performance data to Elasticsearch.

Benefits:

- Application Performance Monitoring
- Bottleneck Detection
- Faster Issue Resolution

7.7 Centralized Log Management

All application logs are collected and stored in a centralized location.

Advantages:

- Single Source of Truth
- Easier Log Search
- Faster Incident Investigation
- Historical Analysis

Administrators can search logs using filters such as:

- Application Name
- Log Level
- Timestamp
- Kubernetes Namespace

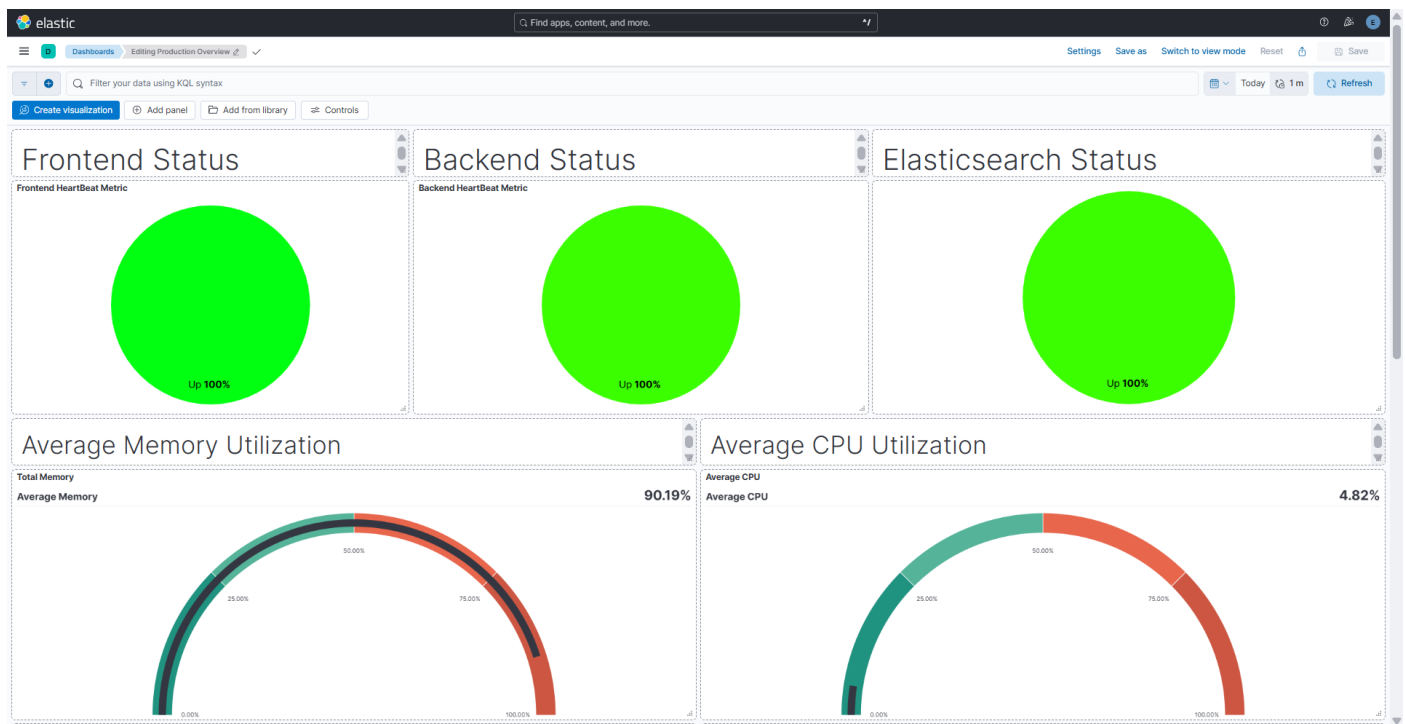


Figure 19: Kibana Dashboard Prod Overview

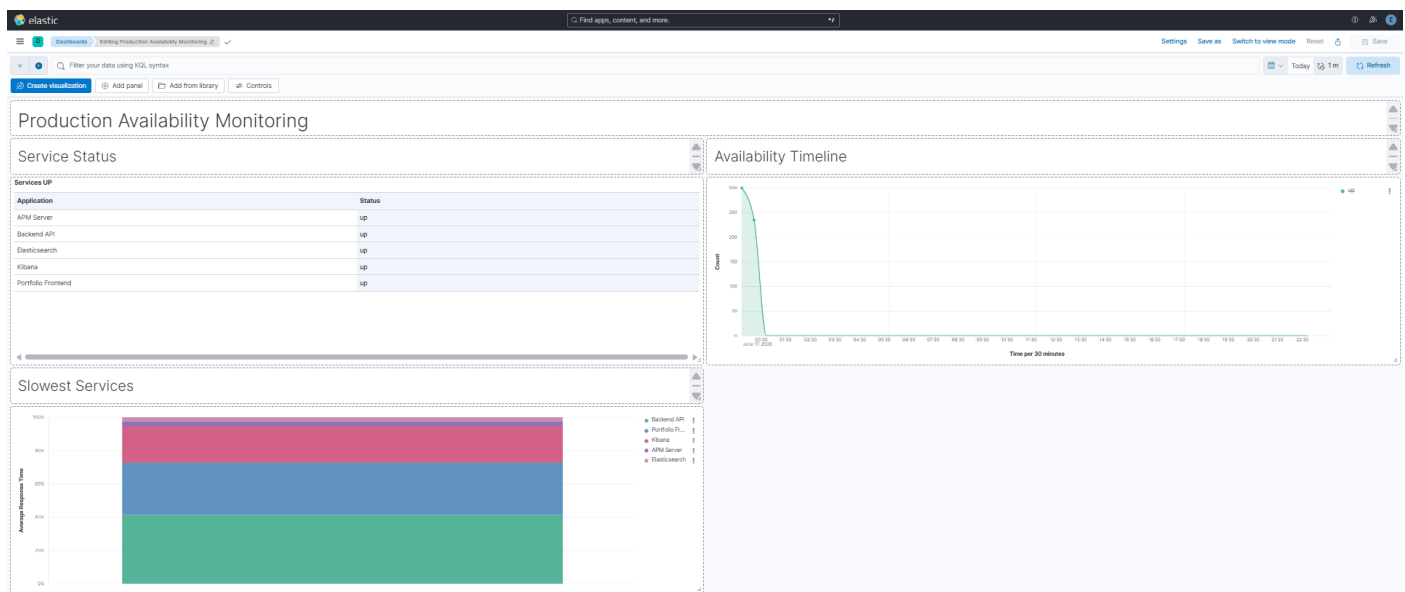


Figure 20: Kibana Dashboard Prod Availability

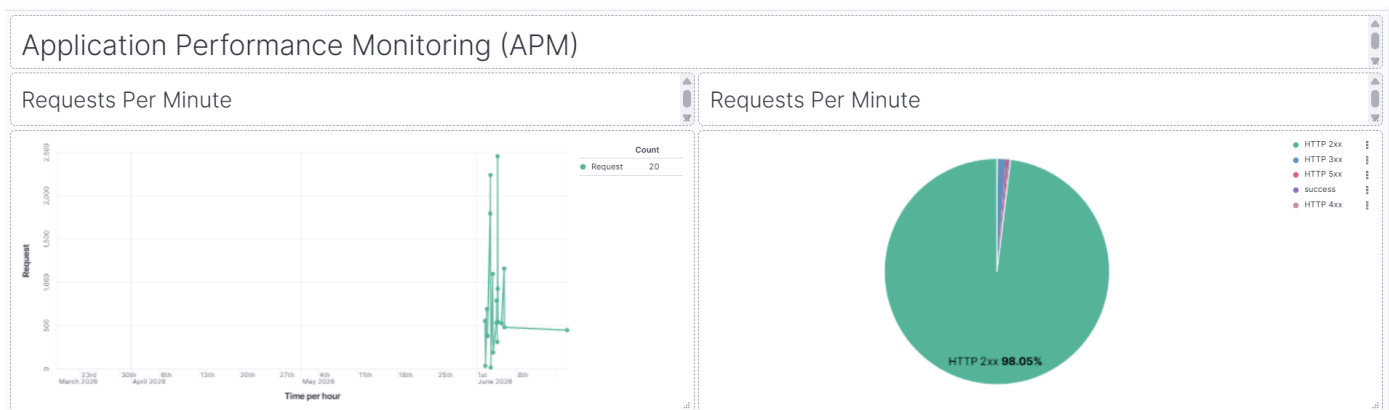


Figure 21: Kibana Dashboard APM - a

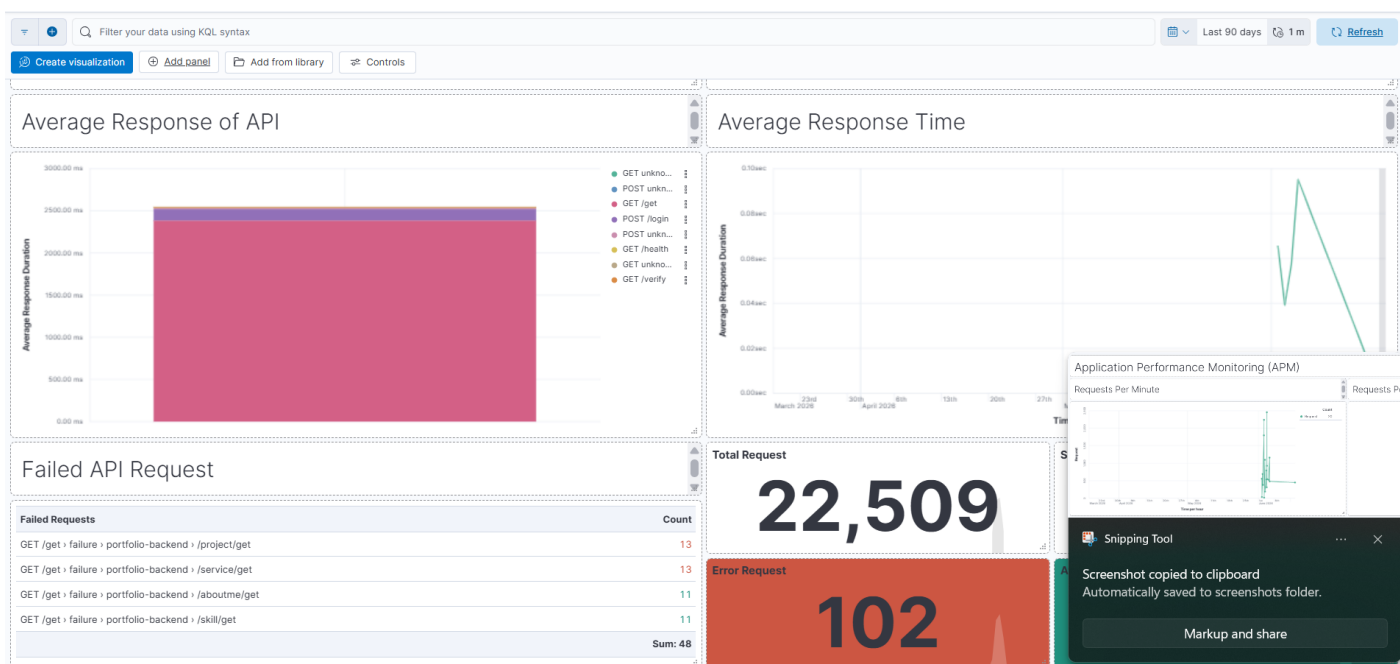


Figure 22: Kibana Dashboard APM - b

7.8 Observability Benefits

The monitoring platform provides:

- Real-Time Visibility
- Faster Troubleshooting
- Centralized Monitoring
- Performance Optimization
- Error Tracking
- Infrastructure Monitoring

This observability solution significantly improves operational efficiency and application reliability.

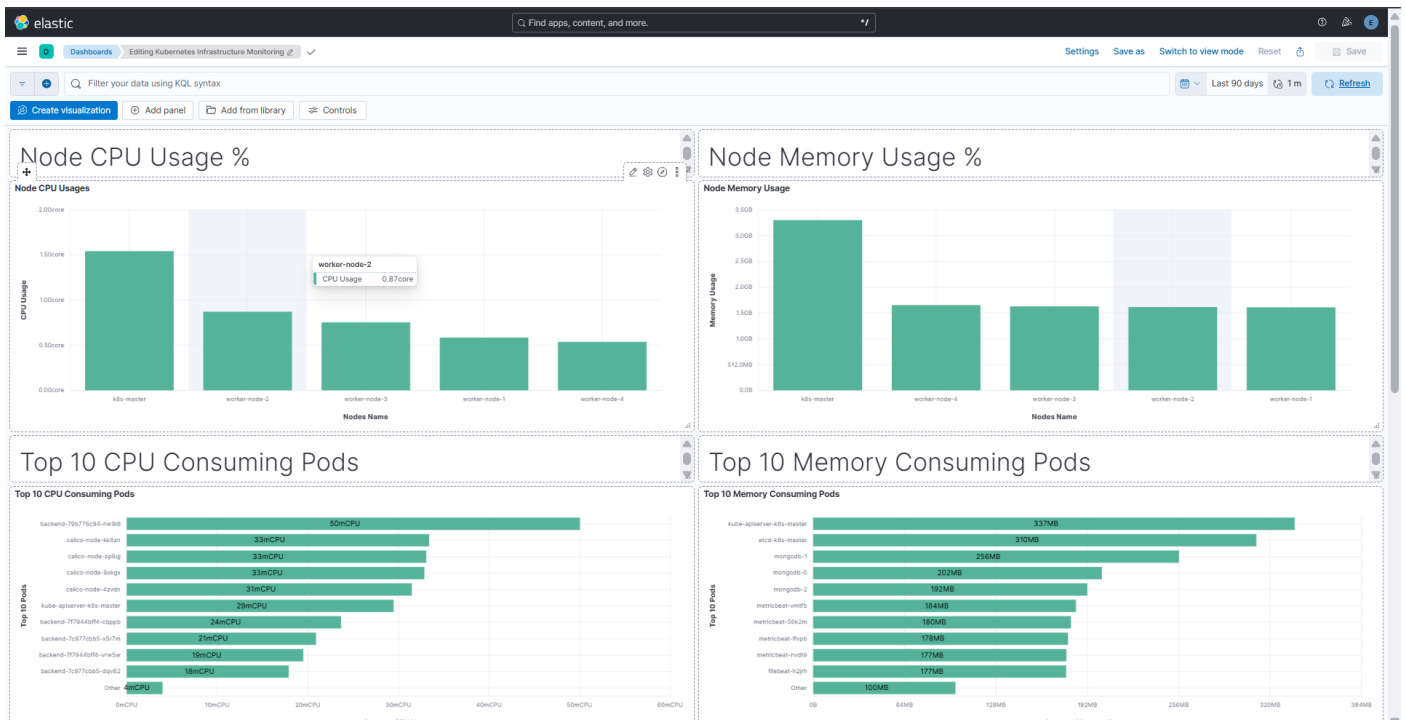


Figure 23: Kibana Dashboard Usages

CHAPTER 7: RESULTS AND CONCLUSION

8.1 Project Results

The project successfully implemented a production-grade MERN stack application deployment using modern DevOps practices and cloud-native technologies.

The solution includes:

- React Frontend Application
- Node.js Backend API
- MongoDB Replica Set Database
- Kubernetes Orchestration Platform
- Jenkins CI/CD Automation
- Docker Containerization
- ELK Monitoring Stack
- HTTPS Security using Let's Encrypt

The deployed infrastructure provides high availability, scalability, automated deployment, and centralized monitoring.

8.2 Key Achievements

Containerization

Frontend and Backend applications were successfully containerized using Docker, ensuring consistency across environments.

Kubernetes Deployment

Applications were deployed on Kubernetes with:

- Deployments
- StatefulSets
- Services
- Ingress
- Horizontal Pod Autoscaler

This provided automated workload management and high availability.

Database High Availability

MongoDB Replica Set was implemented with three members to ensure redundancy and fault tolerance.

CI/CD Automation

Jenkins pipelines automate:

- Source Code Checkout
- Docker Image Build
- Health Verification
- Docker Hub Push

- Image Distribution
- Kubernetes Deployment

This significantly reduced manual deployment effort.

Monitoring and Observability

The ELK Stack provides:

- Centralized Logging
- Infrastructure Monitoring
- Application Performance Monitoring
- Real-Time Visualization

This improves operational visibility and troubleshooting capabilities.

8.3 Benefits Achieved

The implemented solution provides:

- Automated Deployment Process
- High Availability
- Scalability
- Centralized Monitoring
- Secure HTTPS Communication
- Faster Troubleshooting
- Reduced Manual Intervention
- Improved Application Reliability

8.4 Challenges Faced

During implementation, several challenges were encountered and resolved:

- Kubernetes image deployment issues
- MongoDB Replica Set configuration
- Elastic APM integration
- Ingress and SSL configuration
- Jenkins pipeline automation
- Container image distribution to worker nodes

Resolving these issues provided practical experience in real-world DevOps operations.

8.5 Future Enhancements

The project can be extended with:

- AWS EKS Migration
- Terraform Infrastructure Automation
- GitOps using ArgoCD
- Prometheus and Grafana Integration
- Multi-Region Deployment
- Disaster Recovery Automation

- Advanced Security Monitoring

These enhancements can further improve scalability, automation, and resilience.

8.6 Conclusion

This project successfully demonstrates the implementation of a complete production-grade DevOps ecosystem for a MERN stack application.

By combining Docker, Kubernetes, Jenkins, MongoDB Replica Sets, and the ELK Stack, the solution delivers automated deployment, scalability, fault tolerance, monitoring, and operational efficiency.

The project provided practical experience in cloud-native application deployment and modern DevOps practices while creating a reliable and maintainable production environment.